

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING**

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer and Electronics Engineering



**A FINAL REPORT ON
Nepali Handwritten Word Digitization Using Transformer
Model**

*Submitted in partial fulfillment of the requirements for the degree in
Bachelor of Computer Engineering*

Submitted by

Anjila Dangol
Arpita Rimal
Ayusha Shrestha
Sneha Chaulagain

KCE079BCT010
KCE079BCT012
KCE079BCT017
KCE079BCT042

Under the Supervision of

Er. Anish Shilpakar
Data Engineer
Leapfrog Technology

Khwopa College of Engineering
Libali, Bhaktapur
April, 2026

CERTIFICATE OF APPROVAL

This is to certify that this minor project work entitled “**Nepali Handwritten Word Digitization Using Transformer Model**”, submitted by Anjila Dangol (KCE079BCT010), Arpita Rimal (KCE079BCT012), Ayusha Shrestha (KCE079BCT017) and Sneha Chaulagain (KCE079BCT042) has been examined and accepted as the partial fulfillment of the requirements for the degree of Bachelor in Computer Engineering. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

.....
Er. Pratik Timalshena
External Examiner
Senior Lecturer
Department of Electronics and
Computer Engineering
Sagarmatha Engineering College

.....
Er. Anish Shilpakar
Project Supervisor
Data Engineer
Leapfrog Technology

.....
Er. Dinesh Gothe
Head of Department
Department of Computer and Electronics Engineering
Khwopa College of Engineering

COPYRIGHT©

The author has agreed that the library of Khwopa College of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the project work recorded herein or, in their absence, by the Head of the Department where the project report was completed. It is understood that recognition will be given to the author of the report and to the Department of Computer and Electronics Engineering, KhCE, for any use of the material in this project report. Copying, publication, or any other use of this report for financial gain without the approval of the department and the author's written permission is prohibited. Requests for permission to copy or make any other use of the material in this report, in whole or in part, should be addressed to:

Head of Department

Department of Computer and Electronics Engineering
Khwopa College of Engineering (KhCE)
Libali-08, Bhaktapur, Nepal

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our supervisor of the project, **Er.Anish Shilpakar**, for his immense support and guidance throughout the course of this project. His insightful advice, expertise, and unwavering support have been crucial to our work's successful completion. This project would not have been possible without his guidance and mentorship.

We would also like to extend gratitude to our teachers **Er.Mukesh Kumar Pokharel** and **Er.Anish Baral** for their constructive feedback and unwavering support throughout the project. The expertise and analytical recommendations gave us fresh viewpoints, which were crucial for improving our project.

A special thanks to **Er.Dinesh Gothe** for his encouragement and assistance. We have been greatly inspired by his encouragement, which has kept us dedicated and focused on the project.

Furthermore, we are grateful to our institution faculty members, and peers for providing us with the necessary resources, facilities, and a conducive learning environment that greatly contributed to our work.

Finally, we would like to thank everyone who has directly or indirectly assisted us in initiation of our project.

Anjila Dangol	KCE079BCT010
Arpita Rimal	KCE079BCT012
Ayusha Shrestha	KCE079BCT017
Sneha Chaulagain	KCE079BCT042

ABSTRACT

Devanagari is the primary script used for writing the Nepali language and contains complex character structures, conjunct consonants, and vowel modifiers that make handwritten text recognition a challenging task. This project aims to develop a Transformer-based Optical Character Recognition (OCR) system for recognizing handwritten Nepali words and converting them into digital text. The proposed system adopts a TrOCR-inspired architecture that combines a pretrained Vision Transformer (ViT) encoder with a lightweight custom BERT-based Transformer decoder. The ViT encoder extracts contextual visual representations from handwritten word images, while the decoder, designed with a compact Transformer configuration and cross-attention mechanism, generates the corresponding Nepali word sequence in Unicode format. Both components are integrated using HuggingFace's VisionEncoderDecoderModel framework to enable end-to-end handwritten word recognition. Experimental results demonstrate that the proposed model achieves a Character Error Rate (CER) of 5.43%, a Word Error Rate (WER) of 16.58%, and an overall accuracy of 83.42%, indicating its effectiveness in recognizing complex handwritten Nepali text. The developed system provides a promising approach for digitizing handwritten Nepali documents and supports the preservation and accessibility of Nepali written resources.

Keywords: *Nepali Handwritten Word Recognition, Optical Character Recognition, Vision Transformer encoder, TrOCR, BERT, Devanagari Script.*

Contents

List of Tables

List of Figures

List of Symbols and Abbreviation

Abbreviation	Full Form
AdamW	Adaptive Moment Estimation with Weight Decay
AI	Artificial Intelligence
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
BOS	Beginning of Sequence
CER	Character Error Rate
CLS	Classification Token
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
CUDA	Compute Unified Device Architecture
EOS	End of Sequence
FFN	Feedforward Neural Network
fp16	16-bit Floating Point
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
HTR	Handwritten Text Recognition
HWR	Handwritten Word Recognition
IIIT	International Institute of Information Technology
LSTM	Long Short-Term Memory
MHA	Multi-Head Attention
ML	Machine Learning
NED	Normalized Edit Distance
NFC	Unicode Normalization Form C
NHWDTM	Nepali Handwritten Word Digitization Using Transformer Model
OCR	Optical Character Recognition
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SDLC	Software Development Lifecycle
SVM	Support Vector Machines
TrOCR	Transformer-based Optical Character Recognition
TTA	Test-Time Augmentation
UI	User Interface
ViT	Vision Transformer
WER	Word Error Rate

CHAPTER 1

INTRODUCTION

1.1 Background Introduction

Handwritten text recognition (HTR) has gained significant attention in recent years due to its wide applications in document digitization, archival preservation, and human-computer interaction. Despite notable progress in optical character recognition (OCR) for printed text, recognizing handwritten scripts remains a challenging problem, particularly for complex writing systems such as Devanagari. Devanagari, the script used for the Nepali language and several other South Asian languages, exhibits high structural complexity. It consists of base characters, vowel modifiers (matras), conjunct consonants, and compound symbols formed by combining multiple glyphs into a single unit. This complexity is further amplified in handwritten form due to variations in individual writing styles, cursive connections, distortions, and inconsistent spacing between characters and words. These characteristics make segmentation and recognition tasks significantly more difficult compared to Latin-based scripts.

Existing handwritten word recognition (HWR) systems for Nepali and Devanagari scripts have primarily relied on handcrafted feature extraction methods or convolutional neural networks (CNNs) [?, ?, ?]. While CNN-based approaches have shown promising results in capturing local spatial features, they often struggle to model long-range dependencies and sequential context within text, which are crucial for accurate word-level recognition [?]. Furthermore, the scarcity of large-scale, diverse, and publicly available Nepali handwriting datasets limits the generalization capability and robustness of these models [?].

Recent advances in deep learning, particularly Transformer-based architectures, have demonstrated superior performance in various vision and sequence modeling tasks. Vision Transformers (ViT) [?] and Transformer-based OCR models such as TrOCR [?] have shown strong capability in capturing global contextual relationships within images, making them well-suited for handwritten text recognition. Several recent studies have explored the application of Transformers for Indic scripts and multilingual OCR tasks, reporting improved accuracy over traditional CNN-based approaches. [?, ?, ?].

These developments highlight the need for more robust and context-aware models for handwritten Nepali word recognition and digitization. In this work, we explore the use of Transformer-based Optical Character Recognition (TrOCR)-based ar-

chitectures to effectively capture both visual features and contextual dependencies, aiming to improve performance for handwritten Nepali text and their digitization.

1.2 Motivation

Recognizing handwritten Nepali text is challenging due to the complexity of the Devanagari script, which includes compound characters, vowel diacritics, and cursive variations. Manual digitization is time-consuming and error-prone, while existing systems based on handcrafted features or CNNs often fail to capture long-range dependencies and contextual relationships. Limited availability of large, diverse Nepali handwriting datasets further restricts model performance. Recent advances in Transformer-based architectures, such as TrOCR, provide an opportunity to develop an end-to-end system that can accurately recognize handwritten Nepali words, supporting efficient digitization and preservation of Nepali textual resources.

1.3 Problem Statement

Nepali handwritten word recognition using Transformer-based models faces several key challenges:

- **Structural complexity:** The Devanagari script contains conjunct consonants, vowel modifiers, and compound characters, making recognition difficult [?, ?].
- **Handwriting variation:** Diverse writing styles reduce segmentation and recognition accuracy [?].
- **Limitations of existing methods:** CNN and handcrafted approaches capture local features but fail to model long-range dependencies and sequence context effectively [?, ?].
- **Limited datasets:** A lack of large and diverse Nepali handwriting datasets restricts generalization [?, ?, ?].

Although Transformer-based OCR models like TrOCR and Vision Transformers improve global context learning, they still struggle with the linguistic complexity of Nepali handwritten text, highlighting the need for a robust end-to-end OCR system in Unicode format [?, ?, ?].

1.4 Objective

The main objective of this project is:

- To design and develop a Transformer-based OCR system that accurately recognizes and digitizes handwritten Nepali words in the Devanagari script.

1.5 Scope of Project

The scope of this project includes:

- Developing an end-to-end Transformer-based OCR system for recognition of handwritten Nepali words in Devanagari script.
- Enhancing model generalization to handle diverse handwriting styles, complex conjunct consonants, and compound characters.
- Facilitating digitization, preservation, and accessibility of handwritten Nepali documents for educational, cultural, and research purposes.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Review

The area of Handwritten Text Recognition (HTR) has witnessed an enormous shift from the tedious process of manual feature engineering to the era of deep learning, and now, to the era of advanced Transformer-based models. Initial attempts in digitizing Nepali handwriting scripts often encountered roadblocks due to the script’s structural complexities. Dense compound character sets, floating vowel modifiers (matras), and the highly unpredictable nature of cursive handwriting are some of the challenges.

A significant milestone was reached with the advent of Convolutional Neural Networks (CNNs), as these networks are capable of learning and extracting such complex features [?, ?, ?]. Although CNNs proved to be highly efficient in recognizing individual spatial patterns, they faced a structural limitation in incorporating the sequential patterns and long-range dependencies necessary for smooth reading of connected words. This, in turn, led to the natural progression toward sequence-to-sequence models and, consequently, Transformers. Using the self-attention mechanism, Transformers are able to bypass the need for a recurrent structure, as they can consider the entire context of a word. In fact, Vision Transformers and TrOCR models are currently achieving state-of-the-art results, proving to be highly efficient in modeling the complex visual geometry of low-resource languages like Nepali [?, ?, ?].

2.1.1 CNN-based Handwriting Recognition

CNNs were undoubtedly a huge step forward for HTR, as they circumvented the need for feature extraction and instead learned hierarchical spatial features on their own. For example, Baral had built a good foundation with a CNN-based system that was specifically designed for isolated word recognition in Nepali scripts [?]. Ghimire et al. had also further improved this by optimizing the preprocessing steps to be better suited for handling Devanagari scripts [?], and Acharya et al. had also used similar CNN approaches for converting handwritten Nepali scripts to digital text [?]. However, there was a catch. While these models were extremely effective for isolated word recognition, standard CNNs did not possess the “memory” that is often required for handling global dependencies in connected handwriting. This meant that it was extremely sensitive to the stylistic variations in handwriting.

2.1.2 Sequence Modeling and Indic Script OCR

To overcome the blind spots of CNNs on the word level, naturally, the solution was to resort to sequence modeling. For instance, Michael et al. demonstrated how important sequence-to-sequence models can be in cases where you have to directly map unsegmented image data to output text [?].

When we consider the Indic scripts in more detail, the early Nepali-based OCR systems heavily relied on hybrid approaches that combined template matching and feature extraction. However, these were perfectly adequate for simple and well-structured characters but failed miserably when it came to the complex and non-uniform compound characters. The greatest barrier to deeper sequential learning was the absence of data. Gongidi and Jawahar alleviated this problem to some extent by introducing the IIIT-HW-Dev dataset and providing the long-awaited benchmark for Devanagari word-level OCR [?]. Following this, Lalitha et al. attempted to explore new approaches that could improve the accuracy of Indic HTR systems, further supporting the claim that cursive Devanagari needs highly robust models [?].

2.1.3 Vision Transformers for Handwritten Text

Transformers have revolutionized the way we think about handwriting recognition. They have rewritten the rules. It began with Dosovitskiy et al.’s [?] Vision Transformer (ViT) model, which showed it is possible to process image patches with self-attention mechanisms and achieve superior performance compared to traditional convolutional networks. This is something we have seen being applied in handwriting recognition. In fact, Parres and Paredes [?] used vision-based encoder-decoder models to recognize complex, cursive scripts in historical documents. On the other hand, Yajima et al. [?] used Vision Transformers to recognize text in images and also classify script styles. More recently, Li et al. [?] took it to another level with HTR-VT, which uses Vision Transformers in conjunction with sharpness-aware minimization to achieve state-of-the-art performance. In fact, across the board, comprehensive reviews of handwriting recognition models [?] indicate that Transformer models are the new gold standard in extracting text from images.

2.1.4 TrOCR and End-to-End Recognition

A pivotal innovation in the field is the development of TrOCR by Li et al. [?]. This architecture utilizes pre-trained models for both image and text modalities in an end-to-end fashion. The significance of TrOCR lies in its removal of complex CNN

backbones and the reliance on Connectionist Temporal Classification (CTC) loss functions, opting instead for pure self-attention and cross-attention mechanisms. The flexibility of this model has been demonstrated in diverse applications, such as the handwriting pattern recognition research conducted by Adeli Shamsabad and Suen [?].

For low-resource languages, Hasan et al. proposed a Transformer-based model specifically designed for Nepali and Bengali OCR, achieving significant accuracy improvements with a reported Word Error Rate (WER) of 0.15 [?]. Despite these strides, a notable research gap remains in the adoption of highly compressed, localized TrOCR variants. Specifically, the integration of a custom, lightweight BERT-based decoder engineered to address the unique phonetic and morphological nuances of the Devanagari script has yet to be fully explored for the Nepali language. This study seeks to address this gap.

2.1.5 Role of Language Models and Custom Decoders in HTR

While Vision Transformers excel at extracting spatial and structural features from word images, the ultimate accuracy of an end-to-end HTR system is heavily dependent on the decoder’s capacity for sequence modeling [?]. In TrOCR-style architectures, the decoder functions as a language model, generating character sequences based on visual embeddings and the linguistic context of the target language [?]. Standard TrOCR implementations often utilize pre-trained English models like RoBERTa, which may not adequately capture the complexities of Indic scripts.

In the Devanagari script, relying exclusively on visual features can result in ambiguity, particularly when distinguishing between visually similar characters or tightly packed conjuncts [?]. By employing a localized, script-specific language model such as a custom-configured BERT decoder, it is possible to leverage the statistical probability of character co-occurrence in Nepali. Fine-tuning such a decoder specifically for Devanagari text [?, ?] provides the linguistic intuition necessary to correct visual misclassifications, representing a critical optimization pathway that this project investigates.

2.2 Related Theories

2.2.1 Optical Character Recognition (OCR)

OCR is a technology that detects and converts printed or handwritten text found in digital images into machine-encoded, editable text. OCR systems combine hardware and software to transform physical documents into formats that computers can process and search. The most widely known application of OCR is converting scanned paper documents into editable word-processor files. Modern OCR pipelines typically consist of four stages: image acquisition, preprocessing, character or word recognition, and post-processing. Handwritten text recognition is a particularly challenging subfield of OCR because writing styles, stroke widths, and character spacing vary enormously across individuals.

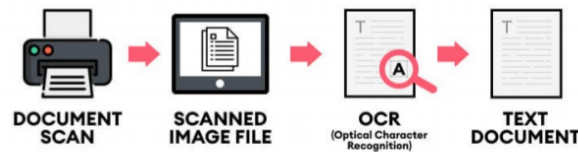


Figure 2.1: General OCR Pipeline

2.2.2 Traditional Approaches to OCR

2.2.2.1 Matrix Matching

Matrix Matching was one of the earliest techniques used in OCR systems. An input character image is partitioned into a fixed grid and each cell is mapped to a pixel value, forming a matrix. This matrix is then compared against a library of pre-stored character templates using a similarity measure. While effective for clean, uniformly typeset text, matrix matching degrades sharply when character fonts, sizes, or orientations deviate from the stored templates, making it unsuitable for handwritten scripts such as Devanagari.

2.2.2.2 Structural and Feature-Based Methods

Structural analysis methods decompose a character into its constituent strokes, loops, junctions, and endpoints. Recognition is achieved by matching these structural primitives against a grammar or rule set that defines each character class. Feature-based methods extract statistical descriptors such as zonal densities, projection profiles, and moment invariants from a segmented character image and feed them

into classical classifiers (e.g., k -NN, SVM, or random forest). These methods offer better tolerance to font variation than matrix matching but still require explicit character segmentation, which is problematic for connected or cursive scripts.

2.2.2.3 Deep Learning-Based OCR Methods

Neural networks have become the dominant paradigm for modern OCR because they can learn hierarchical feature representations directly from raw pixel data without manual feature engineering.

2.2.2.4 Convolutional Neural Networks (CNNs)

CNNs process input images through successive convolutional, pooling, and activation layers that progressively extract spatial features. Given an input image $\mathbf{x}$, a convolutional layer applies a bank of learnable filters $\mathbf{w}_k$:

$$\mathbf{f}_k = \mathbf{x} * \mathbf{w}_k + b_k \quad (2.1)$$

Pooling layers (max or average) then downsample the resulting feature maps, reducing spatial dimensions while retaining dominant activations. The Rectified Linear Unit (ReLU) is the standard non-linearity:

$$\text{ReLU}(x) = \max(0, x) \quad (2.2)$$

Fully connected layers at the top of the network produce a probability distribution over character classes via a softmax output. For isolated character recognition CNNs excel, but recognising full words or lines of text requires a sequential component to capture inter-character context.

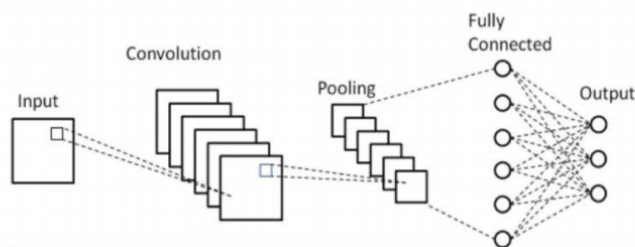


Figure 2.2: Generalised Block Diagram of a CNN Model

2.2.2.5 Recurrent Neural Networks and Long Short-Term Memory

Recurrent Neural Networks (RNNs) extend feed-forward networks to handle sequential data by maintaining a hidden state h_t that is updated at each time step

t :

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b) \quad (2.3)$$

Plain RNNs suffer from vanishing and exploding gradients when sequences are long. Long Short-Term Memory (LSTM) networks address this by introducing gated memory cells. The update equations are:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (2.4)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (2.5)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (2.6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (2.7)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (2.8)$$

$$h_t = o_t \odot \tanh(c_t) \quad (2.9)$$

where f_t , i_t , c_t , and o_t denote the forget gate, input gate, cell state, and output gate respectively; $\odot$ denotes element-wise multiplication. In handwriting recognition, LSTM networks are commonly stacked bidirectionally (BiLSTM) so that each output position aggregates context from both past and future time steps.

2.2.2.6 Connectionist Temporal Classification (CTC)

When a CNN-LSTM encoder produces a sequence of frame-level feature vectors and the corresponding label sequence is shorter (as is typical for word images), alignment between input frames and output characters is unknown during training. CTC resolves this by marginalising over all possible alignments. Given input length T and target sequence $\mathbf{y}$ of length L , CTC introduces a blank token $\emptyset$ and defines the loss as the negative log-likelihood summed over all valid alignments π :

$$\mathcal{L}_{\text{CTC}} = -\ln \sum_{\pi: \mathcal{B}(\pi)=\mathbf{y}} \prod_{t=1}^T p(\pi_t \mid \mathbf{x}) \quad (2.10)$$

where the collapsing function $\mathcal{B}$ removes repeated characters and blanks. CTC enables end-to-end training of sequence-to-label models without requiring frame-level annotations.

2.2.2.7 Transformer Architecture

The Transformer is built entirely on self-attention and dispenses with recurrence. Given an input sequence of vectors $\mathbf{Z} \in \mathbb{R}^{n \times d}$, the scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.11)$$

where Q , K , and V are linear projections of $\mathbf{Z}$. Multi-Head Attention (MHA) runs h independent attention heads in parallel and concatenates their outputs:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2.12)$$

Each Transformer block couples MHA with a position-wise feed-forward network (FFN) and applies layer normalisation with residual connections, enabling gradient flow through very deep networks.

2.2.2.8 Vision Transformer (ViT)

The Vision Transformer (ViT) adapts the Transformer encoder to image inputs by treating the image as a sequence of non-overlapping patches. An image of size $H \times W$ is divided into $N = HW/P^2$ patches of size $P \times P$ pixels. Each patch is flattened and linearly projected to a d -dimensional embedding:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{cls}}; \mathbf{x}_p^1 E; \mathbf{x}_p^2 E; \dots; \mathbf{x}_p^N E] + \mathbf{E}_{\text{pos}} \quad (2.13)$$

where $E \in \mathbb{R}^{(P^2 \cdot C) \times d}$ is the patch embedding matrix and $\mathbf{E}_{\text{pos}}$ is a learnable positional encoding. The sequence $\mathbf{z}_0$ is then processed by L standard Transformer encoder layers. Self-attention across patch tokens allows the model to capture both local stroke features and long-range spatial relationships, which is particularly beneficial for recognising structurally complex scripts like Devanagari/Nepali.

2.2.3 TrOCR: Transformer-Based OCR

TrOCR [?] is an end-to-end transformer-based OCR model that frames text recognition as an image-to-sequence problem. It follows a standard encoder-decoder architecture in which the encoder extracts visual features from the input image and the decoder autoregressively generates the corresponding text sequence.

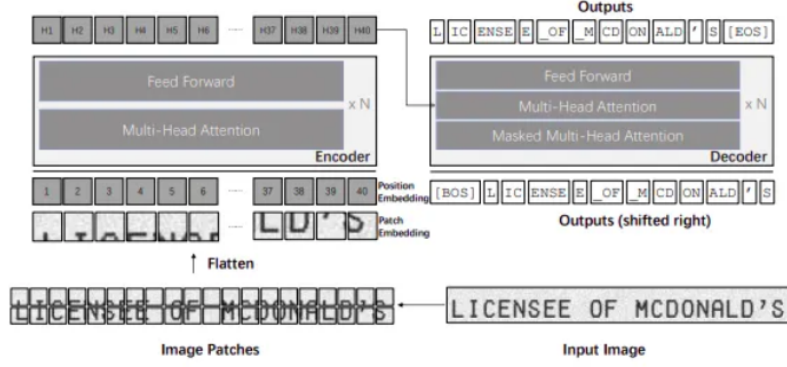


Figure 2.3: TrOCR Architecture

2.2.3.1 Encoder

The encoder is a Vision Transformer (ViT) that processes the input image by first dividing it into a fixed grid of non-overlapping patches. Each patch is linearly projected into a dense embedding vector, and learnable position embeddings are added to retain spatial information. The resulting sequence of patch embeddings is passed through a stack of Transformer encoder layers, each comprising multi-head self-attention and a position-wise feed-forward network. The output is a sequence of contextual hidden states that collectively represent the visual content of the image.

2.2.3.2 Decoder

The decoder is an autoregressive Transformer that generates the output text one token at a time. It is initialised with a beginning-of-sequence token [BOS] and continues generating tokens until the end-of-sequence token [EOS] is produced. Each decoder layer consists of three components: masked multi-head self-attention over previously generated tokens, multi-head cross-attention over the encoder hidden states, and a position-wise feed-forward network. At each decoding step t , the model computes a probability distribution over the vocabulary conditioned on all previously generated tokens and the encoder output $\mathbf{H}$:

$$p(y_t | y_{<t}, \mathbf{H}) = \text{softmax}(W_o \text{TransformerDecoder}(y_{<t}, \mathbf{H})) \quad (2.14)$$

The model is trained by minimising the cross-entropy loss over the full target sequence:

$$\mathcal{L} = - \sum_{t=1}^T \log p(y_t | y_{<t}, \mathbf{H}) \quad (2.15)$$

Both the encoder and decoder are initialised from large pre-trained models, allowing TrOCR to leverage rich visual and linguistic representations prior to task-specific fine-tuning.

2.2.4 Dataset: IIIT-HW-Dev

The IIIT Handwritten Devanagari (IIIT-HW-Dev) dataset provides segmented word images of handwritten Devanagari script along with their Unicode ground-truth transcriptions. The dataset contains samples from multiple writers, covering a diverse vocabulary of Hindi/Nepali words. Each image has been pre-segmented at the word level, making it suitable for word-image-to-text recognition without requiring additional line or word segmentation. The dataset is split into standard training, validation, and test partitions used throughout this work.

2.2.4.1 Challenges of Devanagari Handwriting

Devanagari/Nepali script presents several unique challenges for OCR:

- **Conjunct consonants (half-characters):** Two or more consonants may combine into a single ligature, visually distinct from the base characters.
- **Matras (vowel diacritics):** Vowel signs attach above, below, or beside the base consonant, changing the visual appearance of the character significantly.
- **Headline (Shirorekha):** Most characters share a horizontal headline stroke, causing characters within a word to be visually connected, complicating segmentation-based methods.
- **Inter-writer variability:** Stroke width, slant, and character proportions vary greatly across writers.

The end-to-end encoder-decoder approach adopted in this project directly addresses these challenges by operating at the word-image level and relying on the attention mechanism to learn character boundaries implicitly, without any explicit segmentation step.

2.3 Review Matrix

Table ?? presents a summary of the literature reviewed in this chapter. It consolidates key studies, their methodologies, and contributions, providing a clear overview of the evolution of handwritten text recognition techniques.

Table 2.1: Summary of Literature Review of NHWDTM system

S.N.	Title	Year	Algorithm	Comments
1	Handwritten Nepali character recognition and narration system [?]	2019	CNN	Built a foundational model to identify isolated Nepali characters, bypassing manual feature extraction.
2	Evaluating sequence-to-sequence models for handwritten text recognition [?]	2019	Seq2Seq Models	Proved that sequence modeling is absolutely critical for mapping unsegmented image data directly to text.
3	Nepali handwriting recognition using convolution neural network [?]	2020	CNN, ANN	Optimized preprocessing stages to better handle the structural quirks of Devanagari scripts.
4	An image is worth 16x16 words: Transformers for image recognition at scale [?]	2020	Vision Transformer (ViT)	Introduced ViT, proving self-attention on image patches could outperform traditional convolutions.
5	iiit-indic-hw-words: A Dataset for Indic Handwritten Text Recognition [?]	2021	Dataset Creation	Cleared a major data hurdle by providing a much-needed benchmark for Devanagari word-level OCR.
6	Nepali handwritten character recognition system [?]	2022	CNN	Successfully utilized CNNs to convert handwritten Nepali text into an editable digital format.

Table 2.1 (continued)

S.N.	Title	Year	Algorithm	Comments
7	TrOCR: Transformer-based optical character recognition with pre-trained models [?]	2023	TrOCR (ViT + RoBERTa)	Introduced a brilliant end-to-end architecture that stripped away complex CNN backbones and CTC loss functions.
8	Optical text recognition in Nepali and Bengali: A transformer-based approach [?]	2023	Transformers	Achieved a massive leap in accuracy (WER of 0.15) for low-resource Indic scripts compared to traditional methods.
9	Fine-tuning vision encoder-decoder transformers for historical documents [?]	2023	Vision Encoder-Decoder	Demonstrated the adaptability of vision encoder-decoder models on highly complex, cursive historical handwriting.
10	Text detection and style classification from images Using ViT and transformer decoder [?]	2024	ViT, Transformer Decoder	Utilized Vision Transformers effectively to detect text and classify varied writing styles.
11	HTR-VT: Handwritten text recognition with vision transformer [?]	2025	ViT, SAM	Hit state-of-the-art results by combining Vision Transformers with sharpness-aware minimization.
12	Automated Handwriting Pattern Recognition for mutlti-level personality classification using transformer ocr ... Using TrOCR [?]	2025	TrOCR	Showcased TrOCR's incredible adaptability for complex automated handwriting pattern classification tasks.

Table 2.1 (continued)

S.N.	Title	Year	Algorithm	Comments
13	Transforming images into words: optical character recognition solutions for image text extraction [?]	2025	Comprehensive Review	Reinforced that Transformer architectures are the undeniable new gold standard for image text extraction.
14	Enhancing Accuracy in Indic Handwritten Text Recognition [?]	2026	Deep Learning / HTR	Explored crucial new techniques to handle cursive Devanagari, reinforcing the need for sequence-aware architectures.

CHAPTER 3

METHODOLOGY

3.1 NHWDTM System Workflow Diagram

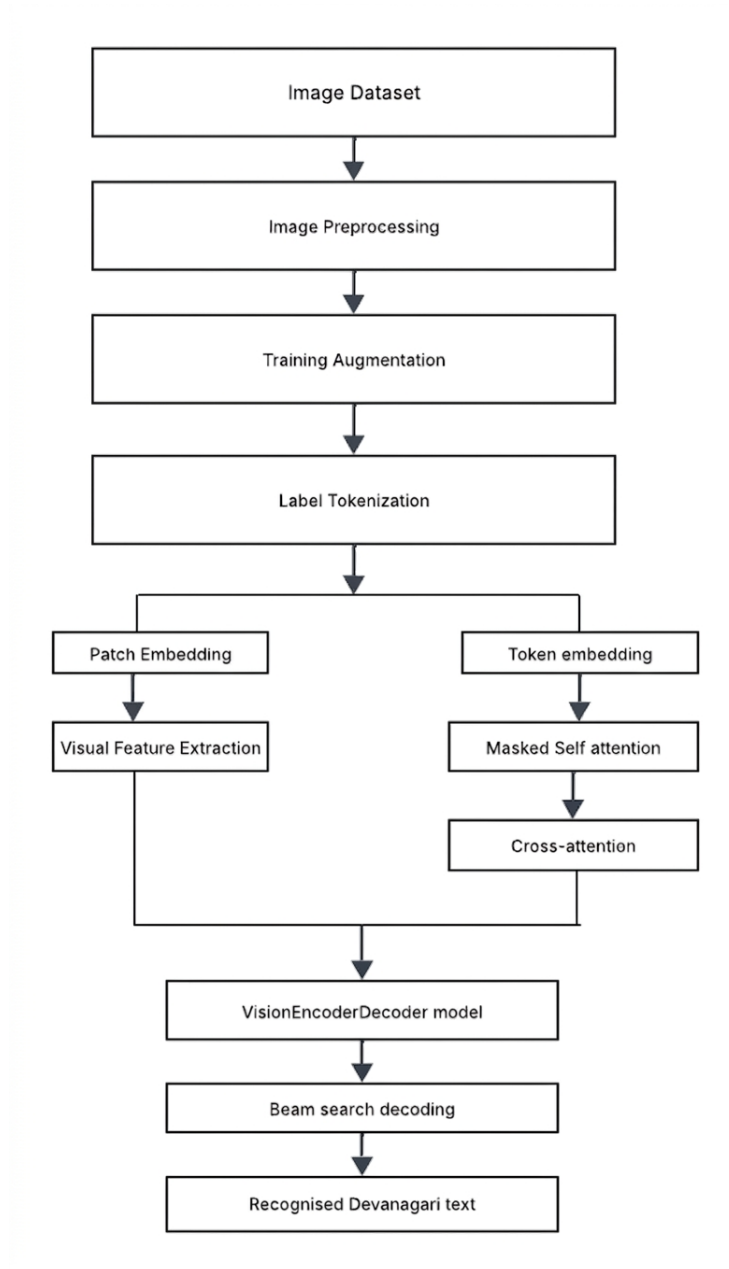


Figure 3.1: Stages of NHWDTM System

3.2 Description of Working Flow of the System

3.2.1 Data Collection:

Initially, the project was focused on character-level recognition for which, a Devanagari Character Dataset [?] was used. And, since matras (diacritics) were not well represented, an additional matra dataset was manually created and curated. However, due to lower accuracy and limitations at the character level, the approach was later shifted to word-level recognition using the IIIT-HW-Dev dataset [?] ?? It contains segmented word-level images along with their corresponding Unicode labels. The dataset was obtained from Kaggle and divided into the following splits:

- **Training set:** 69,802 images
- **Validation set:** 12,713 images
- **Test set:** 12,865 images

Each dataset entry includes an image file and its corresponding Unicode label. In total, 108 unique Devanagari characters were identified, including consonants, matras, and compound characters. The maximum label length observed was 23 characters. This dataset provides a diverse and realistic base for training the handwritten word recognition model.

3.2.2 Image Pre-processing Pipeline:

Each image is passed through a multi-step pre-processing pipeline before being fed to the model. The steps are applied in the following order:

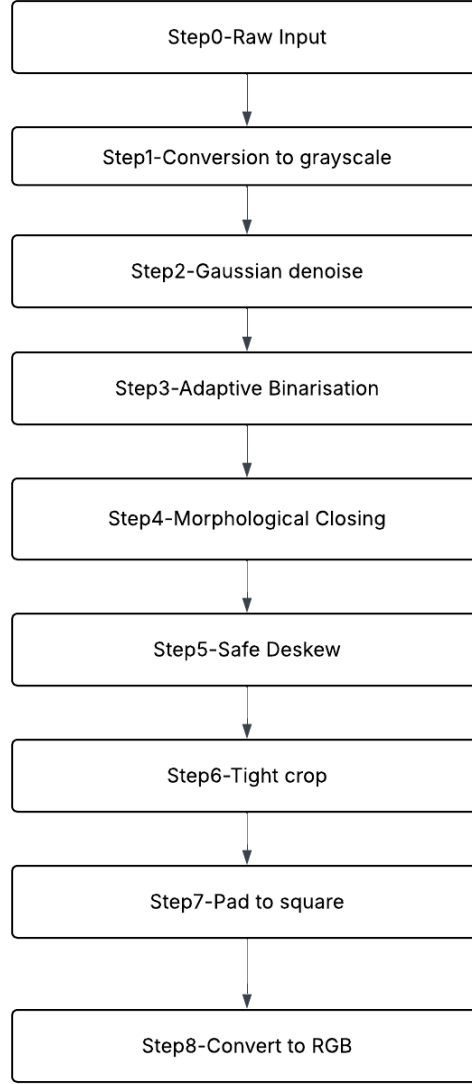


Figure 3.2: Image Pre-processing pipeline

- **Grayscale conversion and denoising:** The image is converted to grayscale and smoothed with a 3×3 Gaussian blur to suppress noise while preserving stroke edges.
- **Adaptive binarisation:** An adaptive Gaussian threshold (block size 25, constant $C = 15$) converts the smoothed image to a binary foreground-background representation. This handles uneven backgrounds more robustly than global Otsu thresholding.

- **Morphological closing:** A 2×2 rectangular kernel is used to close small gaps in Devanagari stroke connections, particularly the shirorekha(headline stroke).
- **Deskewing:** Skew angle is estimated using the minimum area bounding rectangle of foreground pixels. Corrections are applied only for angles in the range $[0.5, 10]$; larger rotations are ignored to avoid misidentifying vertical strokes as tilt.
- **Tight cropping:** The image is cropped to the bounding box of ink pixels with 8 pixels of padding on each side. Crops smaller than 10×10 pixels revert to the uncropped image.
- **Letterbox padding and resize:** The cropped image is centred on a white square canvas, then resized to 128×128 pixels using area interpolation to preserve structural detail.
- **Conversion to RGB and ViT normalisation:** The grayscale image is converted to three-channel RGB and passed to ViTImageProcessor, which rescales it to 384×384 pixels and normalises pixel values to the ViT pre-training distribution (mean = 0.5, std = 0.5 per channel).

All pre-processed images are cached to disk before training to avoid repeated computation. Detailed before/after visuals for each step are provided in Appendices ??-?? .

3.2.3 Training Data Augmentation:

During training only, an additional augmentation pipeline is applied on-the-fly after loading the pre-processed image from cache. The augmentations simulate realistic handwriting variation:

- Gaussian blur ($p = 0.4$, $\sigma \in [0.1, 1.0]$)
- Random affine transform: rotation $\pm 7^\circ$, translation $\pm 7\%$, scale $[0.85, 1.15]$, shear $\pm 5^\circ$
- Colour jitter: brightness ± 0.4 , contrast ± 0.4 , saturation ± 0.1
- Random perspective distortion ($p = 0.3$, distortion scale = 0.15)
- Random autocontrast ($p = 0.2$)
- Random erasing ($p = 0.3$, erased area 2%–12% of image)

Validation and test images are *not* augmented.

3.2.4 Custom Tokenizer Construction:

A character-level tokenizer is built to handle the Devanagari script vocabulary. Key properties are:

- **Vocabulary:** 112 tokens in total ; four special tokens (<pad>, <s>, </s>, <unk>) plus 108 unique Devanagari characters extracted from the training and validation labels.
- **Tokenizer model:** A WordLevel model with Unicode code-point-level pre-tokenisation, treating each character as an independent token.
- **Post-processing:** TemplateProcessing automatically prepends the BOS token <s> and appends the EOS token </s> to every encoded sequence.
- **Decoder:** A Fuse decoder concatenates tokens without whitespace to reconstruct valid Devanagari word forms.
- **Wrapping:** The tokenizer is wrapped in HuggingFace’s PreTrainedTokenizerFast with model_max_length = 64.

3.2.5 ViT Encoder (Visual Feature Extraction):

The encoder is a pre-trained Vision Transformer loaded from microsoft/trocr-base-handwritten. Its architecture is:

- **Patch embedding:** The 384×384 input is divided into non-overlapping 16×16 patches (576 patches), each linearly projected to a 768-dimensional vector.
- **Positional encoding:** A learnable positional embedding is added to each patch, and a special [CLS] token is prepended, giving a sequence of length 577.
- **Transformer layers:** 12 stacked blocks of multi-head self-attention (12 heads, $d_k = 64$), layer normalisation, and a feed-forward network (hidden dim 3072, GELU activation).
- **Output:** A contextual feature matrix of shape $[B, 577, 768]$ encoding rich spatial and semantic information.

3.2.6 Custom BERT-Based Transformer Decoder (Text Generation):

A lightweight BERT-based decoder is built from scratch using BertConfig and BertLMHeadModel. Its actual configuration is:

- **Architecture:** 2 transformer layers, 4 attention heads, hidden size 256, intermediate (FFN) size 512, dropout 0.1, maximum position embeddings 128.
- **Token embedding:** Input token IDs are embedded into 256-dimensional space.
- **Masked self-attention:** Causal left-to-right masking ensures each generated token attends only to prior tokens.
- **Cross-attention:** The decoder attends to the ViT encoder output ($[B, 577, 768]$) as keys and values, with its own hidden states as queries.
- **Output projection:** A linear layer maps hidden states to logits over the 112-token vocabulary.

3.2.7 End-to-End Model Integration:

The encoder and decoder are combined via HuggingFace’s VisionEncoderDecoderModel with following configuration:

- Decoder start token set to `<s>` (BOS).
- Padding token set to `<pad>`; padding positions are masked with -100 in label tensors to exclude them from loss computation.
- Cross-attention enabled via
- Mixed-precision training (fp16) via GradScaler.
- Gradient accumulation over 4 steps (effective batch size 32).

3.2.8 Token Prediction and Beam Search Decoding:

At inference time, the model generates predictions using beam search with the following settings:

- **Beam width:** `num_beams = 4`
- **Candidates returned:** The top 3 scoring sequences are returned as alternative predictions.
- **Termination:** Each beam ends at the `</s>` (EOS) token or the maximum sequence length of 27 tokens.
- **Output selection:** The highest-scoring beam sequence is chosen as the primary prediction.

- **Detokenisation:** Token IDs are decoded back to Unicode Devanagari text; special tokens (`<s>`, `</s>`, `<pad>`) are stripped.
- **NFC normalisation:** Predicted and ground-truth strings are Unicode NFC-normalised before computing CER and WER to avoid spurious character-level mismatches from equivalent code-point sequences.

3.3 Model Architecture

The NHWDTM system adopts a TrOCR-inspired encoder-decoder architecture that pairs a pre-trained Vision Transformer (ViT) with a lightweight custom BERT-based decoder. The two components are integrated using HuggingFace’s Vision Transformer Encoder and Decoder Model framework, enabling end-to-end training with cross-modal attention between the visual and linguistic modalities.

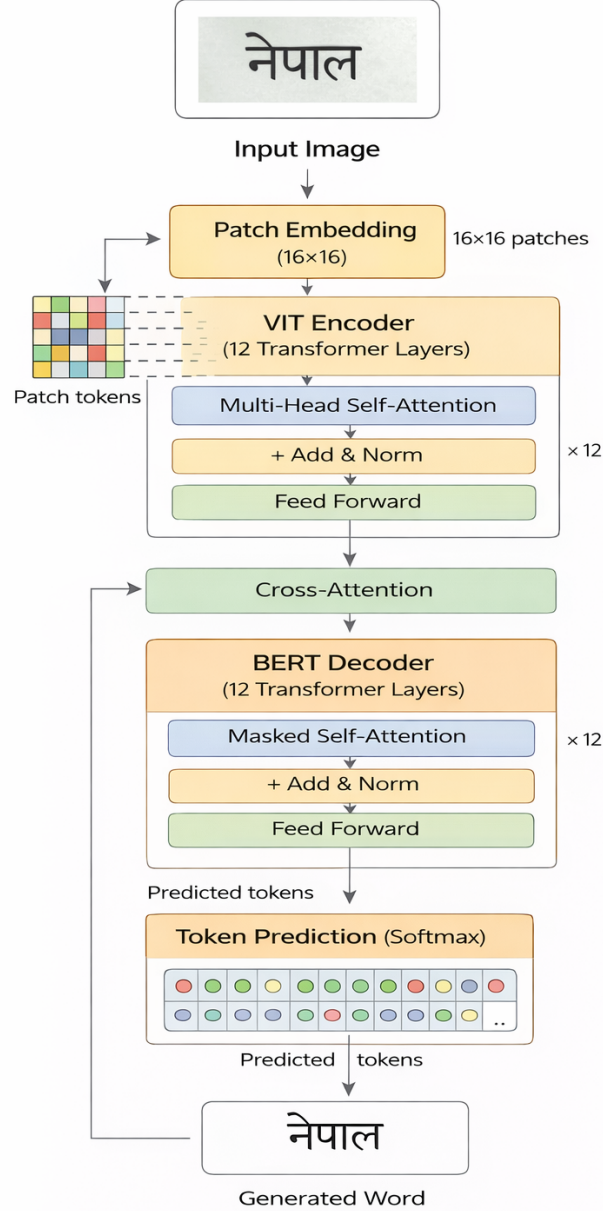


Figure 3.3: Model Architecture of TrOCR EncoderDecoder Model

3.3.1 ViT Encoder

The encoder is a Vision Transformer of the vit-base-patch16-384 variant, with pre-trained weights sourced from microsoft/trocr-base-handwritten; a checkpoint already fine-tuned on handwritten document recognition tasks. This choice provides the model with strong visual priors for cursive and handwritten scripts without requiring training from scratch.

Each input image is resized to 384×384 pixels and partitioned into non-overlapping patches of 16×16 pixels, yielding 576 patches. Each patch is linearly projected into a 768-dimensional embedding vector. A learnable [CLS] token is prepended to the patch sequence, and learnable positional embeddings are added to each position to preserve spatial structure, resulting in a final sequence length of 577.

The sequence is then processed through 12 stacked Transformer blocks. Each block consists of multi-head self-attention with 12 heads ($d_k = 64$ per head), layer normalisation, residual connections, and a position-wise feed-forward network with a hidden dimensionality of 3072 and GELU activation. The encoder produces a contextual feature matrix of shape $[B, 577, 768]$, where B is the batch size. This output encodes rich spatial and semantic information from the handwritten word image and serves as the key-value input for the decoder’s cross-attention mechanism.

3.3.2 Custom BERT-Based Decoder

The decoder is a lightweight, custom-built Transformer decoder constructed using HuggingFace’s BertConfig and BertLMHeadMode. It is intentionally compacted to approximately 4 million parameters for avoiding overfitting on the moderately sized Devanagari training corpus of approximately 70,000 word images.

The decoder comprises 2 Transformer layers, each with 4 attention heads and a hidden dimensionality of 256. The feed-forward sub-layer within each block has an intermediate size of 512 and uses GELU activation. Dropout with probability 0.1 is applied to both attention weights and hidden representations for regularisation. The maximum supported sequence length is 128 token positions.

At each decoding step, the previously generated token identifiers are embedded into a 256-dimensional space via a learned token embedding layer. A causal (left-to-right) masked self-attention mechanism then allows each position to attend only to prior positions, enforcing autoregressive generation. Cross-attention is subsequently applied over the full encoder output, using the encoder’s feature matrix as keys and values and the decoder’s current hidden states as queries. This mechanism enables the decoder to selectively attend to visually relevant regions of the input image at each character generation step.

A final linear projection layer maps the decoder’s hidden states to logits over the 112-token vocabulary, from which the most probable token is selected at each step.

3.3.3 Model Integration

The encoder and decoder are combined into a single trainable model via `VisionEncoderDecoderModel`. The decoder start token is set to `<s>` (BOS), the padding token to `<pad>`, and padding positions are masked with `-100` in the label tensor to exclude them from loss computation. Cross-attention is explicitly enabled in the decoder configuration via the `add_cross_attention` flag.

Table ?? summarises the architectural configuration of both components.

Table 3.1: Model architecture summary

Component	Parameter	Value
ViT Encoder	Input resolution	384×384
	Patch size	16×16
	Sequence length	577 (576 patches + CLS)
	Hidden size	768
	Transformer layers	12
	Attention heads	12
	FFN hidden size	3072
	Pre-trained checkpoint	<code>trocr-base-handwritten</code>
BERT Decoder	Transformer layers	2
	Attention heads	4
	Hidden size	256
	FFN intermediate size	512
	Dropout	0.1
	Max position embeddings	128
	Vocabulary size	112
	Initialisation	Random (from scratch)

3.4 Hyperparameters

The training configuration was designed to accommodate the two-phase training strategy; an initial decoder-only phase followed by full end-to-end fine-tuning while maintaining training stability through mixed-precision arithmetic, gradient accumulation, and cosine annealing scheduling.

3.4.1 Optimiser

The AdamW optimiser was used throughout training, with momentum coefficients $\beta_1 = 0.9$ and $\beta_2 = 0.999$ and a numerical stability constant of $\epsilon = 10^{-8}$.

During Phase 1 (decoder-only training), the encoder parameters were frozen and only decoder parameters were updated, using a learning rate of 5×10^{-4} with weight decay 10^{-3} . During Phase 2 (full fine-tuning), differential learning rates were applied: 5×10^{-7} for the encoder, which was already well pre-trained and required only gentle adaptation, and 5×10^{-5} for the decoder. This differential strategy prevents catastrophic forgetting of the encoder’s pre-trained visual representations while allowing the decoder to continue improving.

3.4.2 Learning Rate Scheduler

A CosineAnnealingWarmRestarts scheduler was applied with a restart period of $T_0 = 10$ epochs, a restart multiplier of $T_{\text{mult}} = 1$, and a minimum learning rate of $\eta_{\text{min}} = 10^{-8}$. The periodic warm restarts help the model escape local minima and explore the loss surface more effectively over longer training runs.

3.4.3 Loss Function

Label-smoothed cross-entropy loss was used with a smoothing coefficient of 0.1. Smoothing distributes a small fraction of the probability mass uniformly across the vocabulary rather than concentrating it entirely on the ground-truth token, which discourages the model from becoming overconfident and improves generalisation to unseen character sequences. Token positions corresponding to padding (label value = -100) are excluded from the loss computation.

3.4.4 Training Efficiency

Mixed-precision training (fp16) was employed via PyTorch’s GradScaler to reduce memory consumption and accelerate computation on GPU hardware. Gradient accumulation over 4 steps was applied, giving an effective batch size of 32 while keeping the per-step memory footprint manageable. Gradient clipping at a maximum norm of 1.0 was applied before each optimiser step to prevent destabilising gradient explosions during early training.

3.4.5 Regularisation and Augmentation

In addition to label smoothing and dropout within the decoder (0.1), on-the-fly data augmentation was applied exclusively to the training set to improve robustness to handwriting variation. The augmentation pipeline comprised Gaussian

blur ($p = 0.4$, $\sigma \in [0.1, 1.0]$), random affine transformations (rotation $\pm 7^\circ$, translation $\pm 7\%$, scale $[0.85, 1.15]$, shear $\pm 5^\circ$), colour jitter (brightness and contrast ± 0.4 , saturation ± 0.1), random perspective distortion ($p = 0.3$, distortion scale 0.15), random autocontrast ($p = 0.2$), and random erasing ($p = 0.3$, erased area 2%–12% of image area).

3.4.6 Early Stopping and Checkpointing

Validation was performed every 2 epochs on the held-out validation set. The best model checkpoint was saved whenever a new minimum validation CER was achieved. Training was terminated early if no improvement was observed for 10 consecutive validation rounds (patience = 10), corresponding to a maximum of 20 epochs without progress.

Table ?? provides a consolidated summary of all hyperparameter values used during training.

Table 3.2: Training hyperparameter summary

Category	Hyperparameter	Value
Optimiser	Algorithm	AdamW
	β_1, β_2	0.9, 0.999
	ϵ	10^{-8}
	Weight decay (Phase 1)	10^{-3}
Learning rate	Phase 1 (decoder only)	5×10^{-4}
	Phase 2 encoder LR	5×10^{-7}
	Phase 2 decoder LR	5×10^{-5}
Scheduler	Type	CosineAnnealingWarmRestarts
	Restart period T_0	10 epochs
	Minimum LR $\eta_{\min}$	10^{-8}
Training	Batch size (per step)	8
	Gradient accumulation	4 steps (effective batch 32)
	Gradient clip norm	1.0
	Max epochs	52
	Validation frequency	Every 2 epochs
	Early stopping patience	10 validation rounds
	Precision	fp16 (mixed precision)
Loss	Function	Cross-entropy
	Label smoothing	0.1
Inference	Decoding strategy	Beam search
	Beam width	4

3.5 Performance Evaluation Metrics

The model was evaluated on the held-out test set of 12,865 word images using the following metrics. All string comparisons were performed after Unicode NFC normalisation of both predicted and ground-truth strings.

3.5.1 Character Error Rate (CER)

CER measures the minimum number of character-level edit operations where S,I and D stands for substitutions (S), deletions (D), and insertions (I) required to transform the predicted string into the ground-truth string, normalised by the total number of characters N in the ground truth. It is computed using the Levenshtein edit distance algorithm. A CER of 0 indicates perfect character-level recognition, and it served as the primary metric for model selection during training.

$$\text{CER} = \frac{S + D + I}{N}$$

3.5.2 Word Error Rate (WER)

WER measures the proportion of test samples for which the predicted transcription does not exactly match the ground-truth word label. Since each sample in the IIIT-HW-Dev dataset corresponds to a single segmented word image, WER directly reflects the rate of completely incorrect predictions.

$$\text{WER} = \frac{\text{Number of incorrectly predicted words}}{\text{Total number of test words}}$$

3.5.3 Exact Match Accuracy

Exact Match Accuracy measures the proportion of test samples for which the model produces a transcription that is identical to the ground-truth label in its entirety. It is the complement of WER and provides an intuitive measure of end-to-end recognition performance.

$$\text{Exact Match Accuracy} = \frac{\text{Number of exactly matched words}}{\text{Total number of test words}} \times 100\%$$

3.5.4 Normalised Edit Distance (NED)

NED provides a length-normalised measure of string similarity between the predicted and ground-truth transcriptions. Unlike CER, which is normalised by the

ground-truth length only, NED normalises by the longer of the two strings, making it more robust when predicted sequences are significantly shorter or longer than the reference. It ranges from 0 (identical strings) to 1 (completely dissimilar strings).

$$\text{NED} = \frac{\text{Levenshtein distance}(\hat{y}, y)}{\max(|\hat{y}|, |y|)}$$

where $\hat{y}$ is the predicted string and y is the ground-truth string.

3.6 Model Variants Explored

In addition to the NHWDTM system architecture described in Section 3.3, three alternative model configurations were trained on the same dataset and under the same conditions to guide design decisions. The proposed model (v3) was selected as the primary configuration based on its superior downstream performance. Table ?? summarises the key architectural differences across all four variants; quantitative results are reported in Section 5.1.

Model	Encoder	Decoder	Dec. Params	Key Difference
Baseline	ViT-Base (<i>trocr-base-handwritten</i>)	RoBERTa (extended Devanagari vocab)	~125M	Standard fine-tuning; no custom decoder
TrOCR-Small	ViT-Small (<i>trocr-small-handwritten</i>)	Custom BERT (2L / 4H / 256 d)	~4M	Lighter encoder; faster training
TrOCR-Large	ViT-Large (<i>trocr-large-handwritten</i>)	Custom BERT (4L / 8H / 512 d)	~13M	Maximum encoder capacity
Proposed TrOCR base with custom decoder	ViT-Base (<i>trocr-base-handwritten</i>)	Custom BERT (2L / 4H / 256 d)	~4M	Best balance of capacity and efficiency

Table 3.3: Model variants explored during development

CHAPTER 4

SYSTEM DESIGN

4.1 Requirement Specification

4.1.1 Functional Requirements

- **Input Handling:** The system accepts handwritten Nepali word images from the IIIT-HW-Dev dataset during training and evaluation, as well as user-provided images during inference.
- **Image Pre-processing:** Input images are automatically passed through a multi-step pipeline comprising grayscale conversion, denoising, adaptive binarisation, morphological closing, deskewing, tight cropping, and letterbox padding to 128×128 pixels, followed by rescaling to 384×384 pixels and normalisation by the ViTImageProcessor to match the ViT encoder’s pre-training distribution.
- **Feature Extraction and Recognition:** The pre-trained ViT encoder extracts a contextual visual feature matrix from the input image, and the custom BERT-based decoder attends to these features via cross-attention to autoregressively generate the corresponding Nepali Unicode word.
- **Tokenization:** A custom character-level tokenizer encodes ground-truth Unicode labels into integer token sequences and decodes predicted token IDs back into Devanagari text using a 112-token vocabulary.
- **Training and Evaluation:** The system is trained using label-smoothed cross-entropy loss and evaluated using Character Error Rate (CER), Word Error Rate (WER), Exact Match Accuracy, and Normalised Edit Distance (NED).
- **Inference:** The model generates predictions using beam search decoding ($\text{num_beams} = 4$), returning the top 3 candidate transcriptions ranked by sequence log-probability score.
- **Output:** The primary output is the highest-scoring candidate Nepali word in Unicode format. For single-image inference, two additional confidence signals are reported:

1. Per-character confidence, expressed as a percentage derived from per-token transition probabilities.
2. Sequence-level beam scores (log-probability) for each of the top 3 alternative predictions, allowing the user to assess and manually select a more appropriate candidate if needed.

4.1.2 Non-Functional Requirement

- **Performance:** The system should provide accurate predictions with reasonable response time using GPU acceleration.
- **Scalability:** The model can be extended to process full sentences or paragraphs by adding a word segmentation module.
- **Accuracy:** The system aims to achieve low CER and WER, and high Exact Match Accuracy on the test dataset.
- **Usability:** The model should be easy to use and deploy as a standalone tool or API for digitizing handwritten documents.
- **Reliability:** The system should produce consistent results across different handwriting styles.

4.2 Use Case Diagram

The Use Case Diagram ?? illustrates the interaction between the actor and the Nepali Handwritten Word Recognition System.

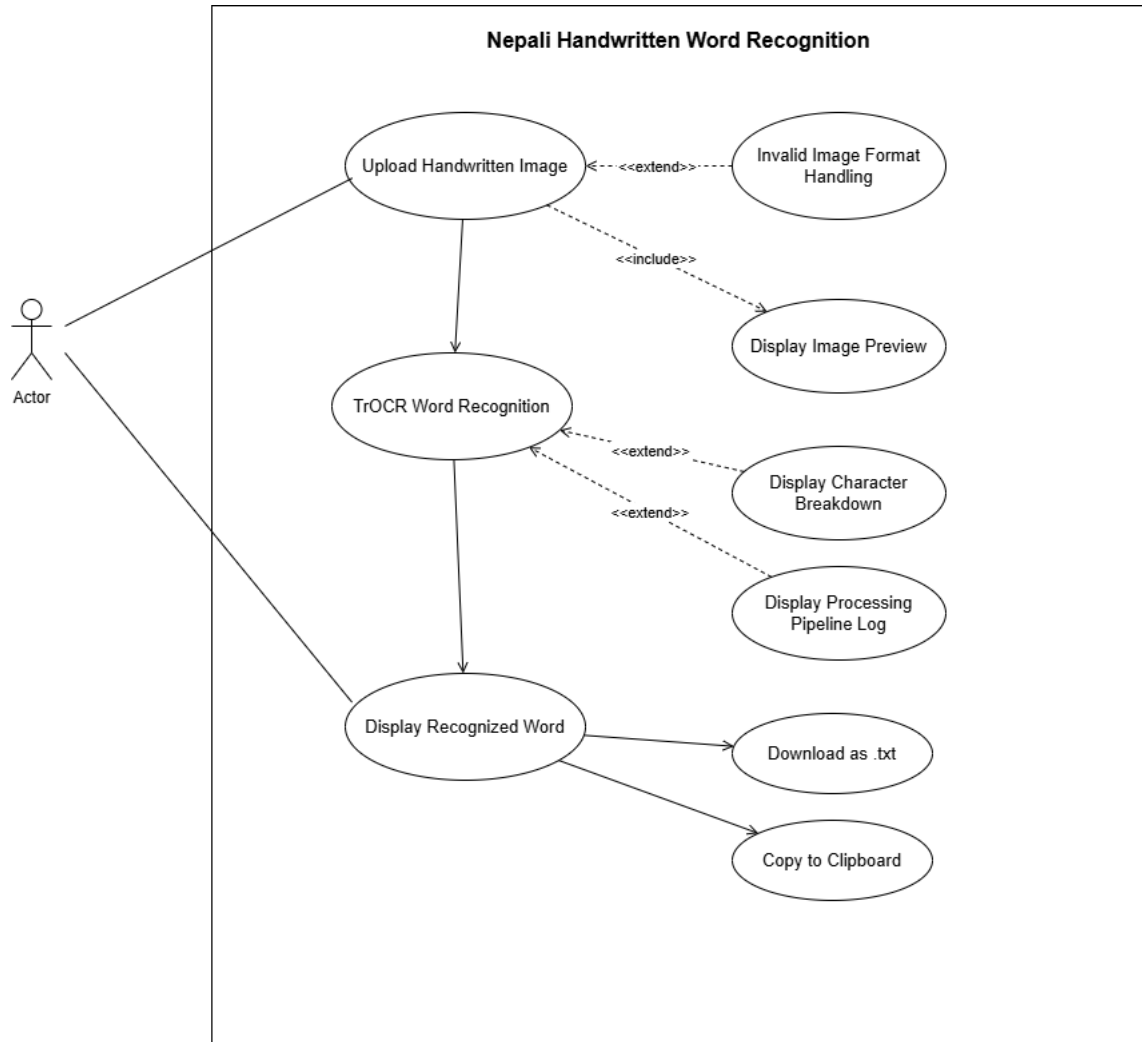


Figure 4.1: Use Case Diagram of Nepali Handwritten Word Digitization

The process begins when the user uploads a handwritten image, which automatically includes displaying an image preview and may extend to handle invalid image formats. The uploaded image is directly passed into the TrOCR Word Recognition process, which may optionally extend to display a character breakdown. Finally, the recognized word is displayed to the user, with additional options to download the result as a .txt file or copy it to the clipboard.

4.3 Class Diagram for System

This class diagram ?? illustrates the structure of the Nepali Handwritten Word Recognition System.

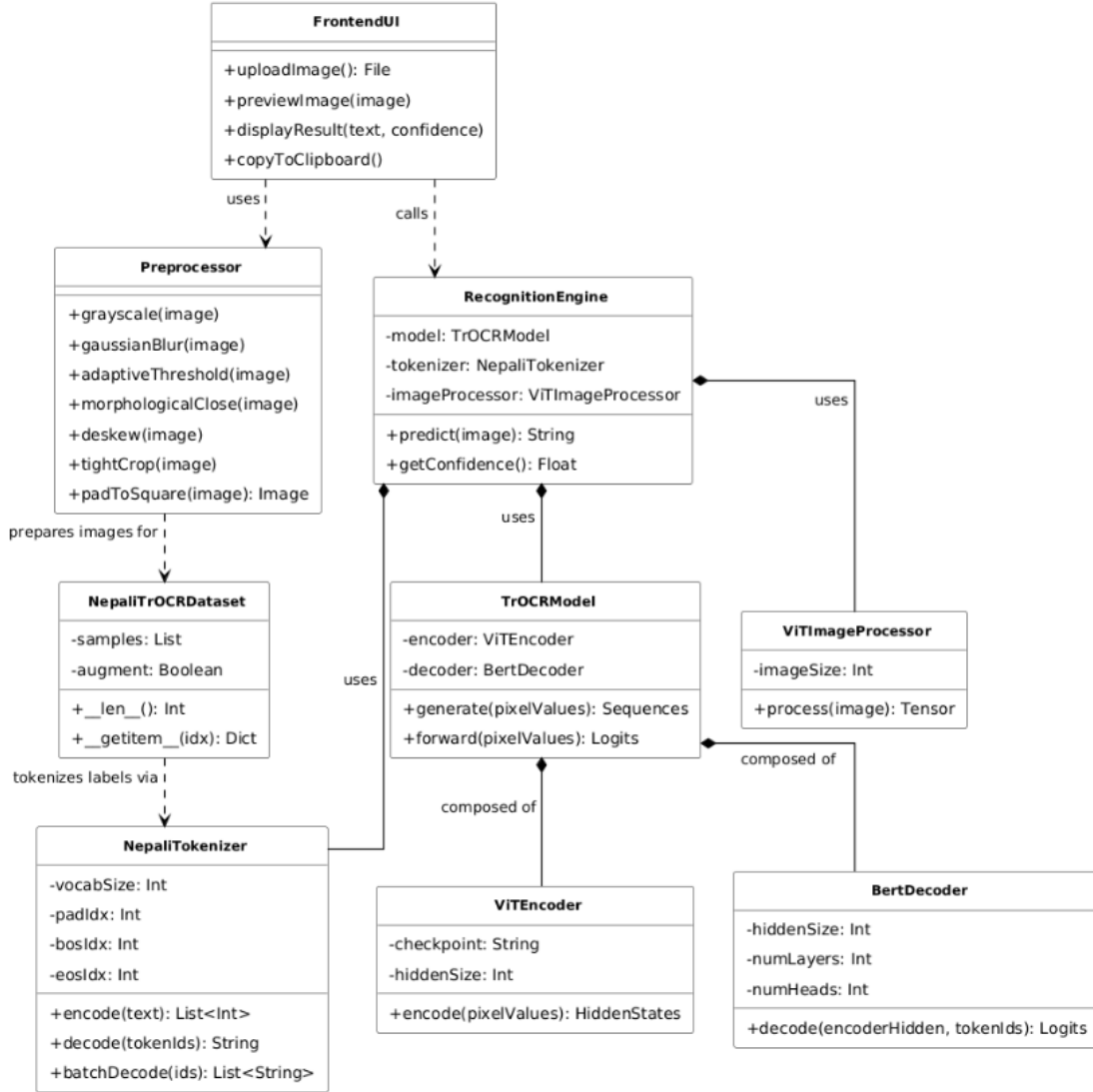


Figure 4.2: Class diagram of Nepali Handwritten Word Digitization

The FrontendUI is the entry point where the user uploads a handwritten word image and receives the predicted text with confidence scores. It directly interacts with the Preprocessor, which cleans the image through a seven-step pipeline, and the RecognitionEngine, which handles inference. The preprocessed images feed into the NepaliTrOCRDataset, which uses the NepaliTokenizer to convert Devanagari labels into character-level token IDs. During inference, the RecognitionEngine coordinates the TrOCRModel, NepaliTokenizer, and ViTImageProcessor. The TrOCRModel is composed of a pretrained ViT encoder and a lightweight custom

BERT-based decoder consisting of two layers, four attention heads, hidden size 256 to generate the recognised character sequence. Each class has a single well-defined responsibility, ensuring a clean data flow from raw handwritten input to Unicode text output.

4.4 Sequence Diagram

The sequence diagram ?? depicts the step-by-step interaction between the system components during a recognition request. The user uploads a handwritten image through the Frontend UI, which previews the image before the user initiates recognition. The frontend sends a POST request to the FastAPI Backend, which forwards the image to the Preprocessor. The Preprocessor applies grayscale conversion, binarization, deskewing, and tight cropping before returning the cleaned image to the backend. The backend then uses the TrOCR Model to generate IDs and the Nepali Tokenizer to decode these into Unicode text. Finally, the results are sent back to the Frontend UI for display to the user.

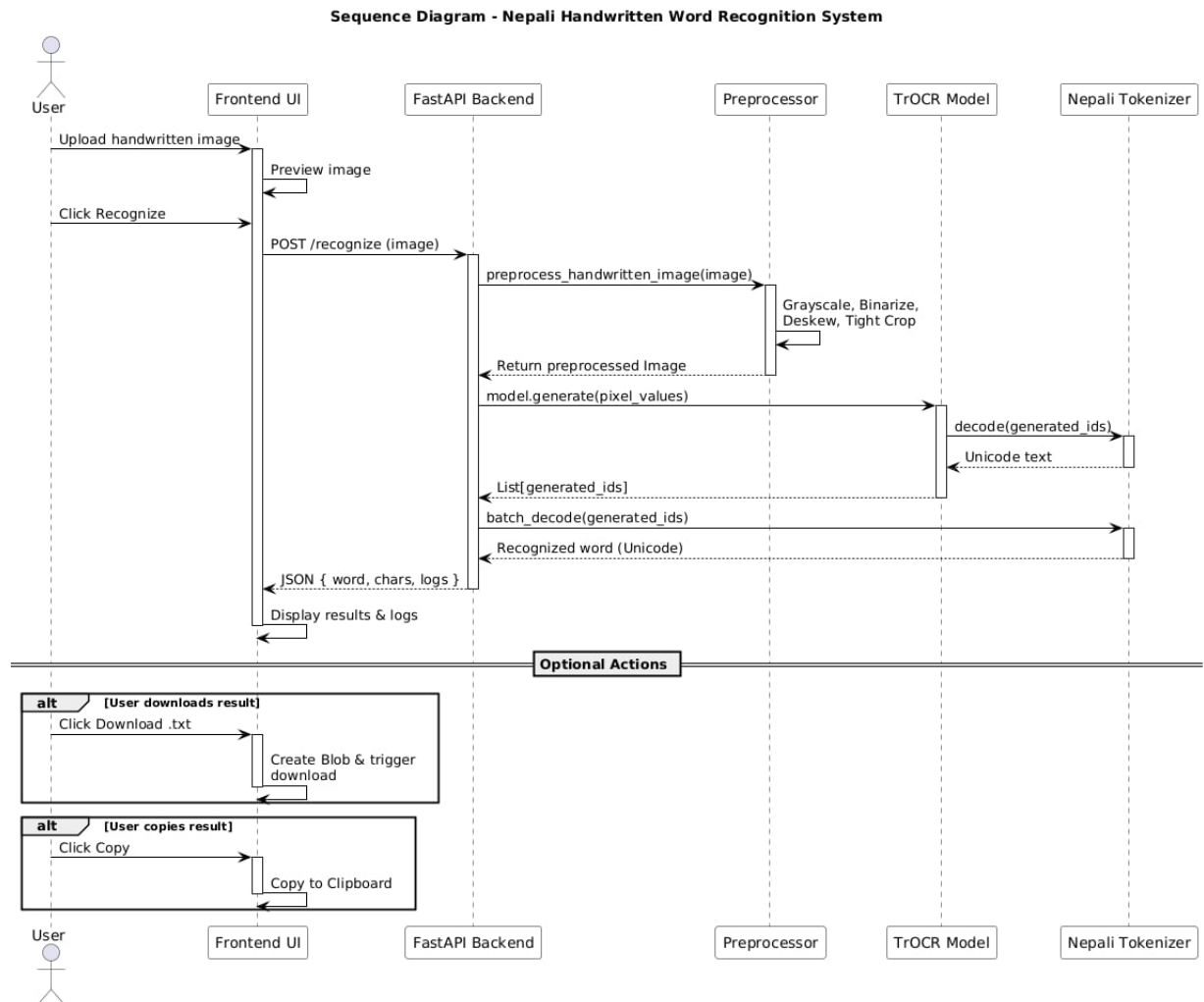


Figure 4.3: Sequence Diagram of Nepali Handwritten Word Digitization

The backend then calls the TrOCR Model to generate token IDs, which are decoded into Unicode text by the Nepali Tokenizer. The final recognized word is returned to the frontend as a JSON response and displayed to the user. Optionally, the user may download the result as a .txt file or copy it to the clipboard.

CHAPTER 5

RESULT AND DISCUSSION

5.1 Experiment Setup

All experiments were conducted on Kaggle’s cloud-based GPU infrastructure. Table ?? summarises the hardware and software environment used throughout training and evaluation.

Component	Specification
GPU	NVIDIA Tesla T4 (16 GB VRAM)
System RAM	29 GB
Operating System	Linux (Ubuntu 20.04)
Python	3.10
PyTorch	2.1 (CUDA 12.1)
HuggingFace Transformers	4.40
Training Precision	fp16 (mixed precision)
Training Platform	Kaggle Notebooks

Table 5.1: Hardware and software environment

5.2 Training Configuration

Four model configurations were explored during the course of this project, each differing in encoder backbone and decoder architecture.

The Baseline model retained the original RoBERTa-based decoder from microsoft/trocr-base-handwritten with the Devanagari character set added to its existing vocabulary. While this required minimal changes, the RoBERTa decoder had been pre-trained on English text, causing it to treat Devanagari characters as rare multi-byte fallback tokens and producing excessively long, fragmented token sequences with poor generation coherence for Nepali words.

The TrOCR-Small variant replaced the ViT-Base encoder with a ViT-Small encoder of approximately 22 million parameters. Although this reduced memory usage and training time, the smaller encoder capacity limited the richness of visual features available for structurally complex conjunct consonants and compound Devanagari characters, resulting in higher recognition errors on difficult word forms. The TrOCR-Large variant used a ViT-Large encoder with 307 million parameters across 24 transformer layers, paired with a larger custom BERT decoder of 4

layers and 8 attention heads. While the larger encoder provided richer visual representations, the substantially increased memory footprint made full Phase 2 fine-tuning impractical on the available Kaggle GPU infrastructure, restricting the number of training epochs and limiting the model’s final performance.

Based on these observations, the Proposed Model (v3) was selected as the primary configuration. It pairs the ViT-Base encoder which provides sufficient visual representation capacity for Devanagari handwriting without the memory constraints of the large variant along with a compact custom BERT-based decoder of only 2 transformer layers, 4 attention heads, and a hidden dimensionality of 256, yielding approximately 4 million decoder parameters. This lightweight decoder was intentionally designed to avoid overfitting on the approximately 70,000 training samples while retaining sufficient capacity for character-level autoregressive generation. Crucially, replacing the English-pretrained RoBERTa decoder with a custom character-level decoder built entirely from the Devanagari vocabulary eliminates the tokenizer mismatch present in the baseline, achieving complete character coverage with zero unknown token occurrences. The proposed model therefore offers the best balance of recognition accuracy, training efficiency, and stability under the available computational constraints, and is the focus of the evaluation presented in this chapter. Quantitative results for all four variants are reported in Table ??.

Training Setting	Value (all variants)
Dataset	IIIT-HW-Dev
Train / Val / Test	69,802 / 12,713 / 12,865 images
Tokenizer	Custom Devanagari (112 tokens)
Loss function	Label-smoothed CE ($\alpha = 0.1$)
Optimiser	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$)
Batch size (effective)	32 (micro-batch 8, accumulation 4)
Mixed precision	fp16
Decoding (inference)	Beam search (<code>num_beams</code> = 4)
Evaluation metrics	CER, WER, Accuracy
Training platform	Kaggle (NVIDIA Tesla T4)

Table 5.2: Shared training configuration across all model variants

Model	Encoder	Decoder	Dec. Prm	CER%	WER%	Acc.%
Baseline	ViT-Base	RoBERTa (ext)	~125M	41.92	52.92	57.49
TrOCR-Small	ViT-Small	Custom (2L/4H)	~4M	13.29	29.7	70.13
TrOCR-Large	ViT-Large	Custom (4L/8H)	~13M	8.08	20.40	79.60
Proposed TrOCR base with custom decoder	ViT-Base	Custom BERT	~4M	5.43	16.58	83.42

Table 5.3: Architectural comparison and test set results across all model variants

5.2.1 TrOCR base Model Results

The training process was conducted in two distinct phases. During Phase 1 (epochs 1–15), the Vision Transformer (ViT) encoder remained frozen, and only the custom decoder which was configured with 2 layers, 4 attention heads, and a hidden dimension of 256 was trained. As a result, the Character Error Rate (CER) initially remained relatively high (approximately 21%) and exhibited fluctuations. This behavior can be attributed to the decoder learning from scratch while relying on fixed visual representations that were not specifically optimized for the target task. The temporary increase in CER observed around epochs 12–13 (rising from 16.8% to 21.4%) reflects a common phenomenon in training dynamics, where the model departs from a local optimum before converging toward a more effective solution.

Phase 2 began at epoch 16, when the encoder was unfrozen and allowed to fine-tune jointly with the decoder. This transition resulted in a noticeable improvement in performance, with CER decreasing sharply from approximately 11.7% to 10% within a few epochs. The improvement is explained by the encoder adapting its feature representations; which was originally trained on English handwriting using TrOCR for better capturing of the structural characteristics of the Devanagari script, including its distinctive stroke patterns and vowel diacritics (matras). Following this adjustment, both encoder and decoder components converged more effectively, leading to a steady decline in CER.

From epoch 16 onward, the gradual and consistent reduction in CER, ultimately reaching a minimum value of 6.68% at epoch 52, indicates stable learning. The absence of abrupt fluctuations during this phase suggests that the learning rate strategy ; Cosine Annealing with Warm Restarts was appropriately conservative. This choice helped maintain stability during fine-tuning, preventing disruption of the pretrained encoder weights while enabling continuous performance improvements.

Figure ?? illustrates the progression of training loss and validation Character Error Rate (CER) across epochs, highlighting the performance improvement following encoder unfreezing at epoch 16.

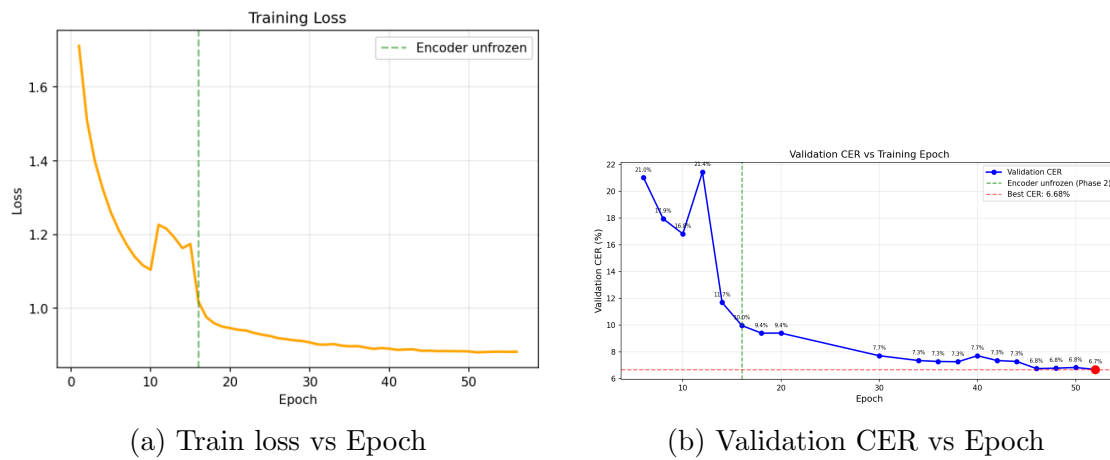


Figure 5.1: Training dynamics across epochs. The green dashed line marks the transition to Phase 2 (encoder unfrozen). The red dashed line indicates the best validation CER of 6.68%.

The distribution of Normalized Edit Distance (NED) is highly right-skewed, indicating that the model performs very well on the majority of test samples while struggling with a small subset of more difficult cases. A total of 10,736 samples, representing approximately 83.5% of the test set, achieved perfect recognition ($\text{NED} = 0$). This large concentration at zero demonstrates that the model is able to accurately predict most words without any character-level errors.

Despite this strong performance, the mean NED is 5.22%, which is slightly elevated due to a small number of challenging samples forming a long, sparse tail in the distribution. These outliers correspond to cases where the model makes significant errors, including instances where predictions are entirely incorrect ($\text{NED} = 100\%$). However, such cases are relatively rare and do not substantially affect the overall performance trend.

Normalized Edit Distance is defined as the edit distance divided by the maximum length of the predicted and reference sequences. Unlike Character Error Rate (CER), which normalizes only by the reference length, NED accounts for both insertions and deletions more symmetrically by penalizing predictions that are excessively longer or shorter than the ground truth. In this evaluation, the NED (5.22%) and CER (5.82%) values are very close, suggesting that the model maintains good control over sequence length. In other words, it is neither frequently generating extra characters nor truncating outputs, which indicates stable and well-calibrated predictions.

Overall, the distribution reflects a model that is highly accurate on most inputs, with only a small proportion of difficult cases contributing to the residual error.

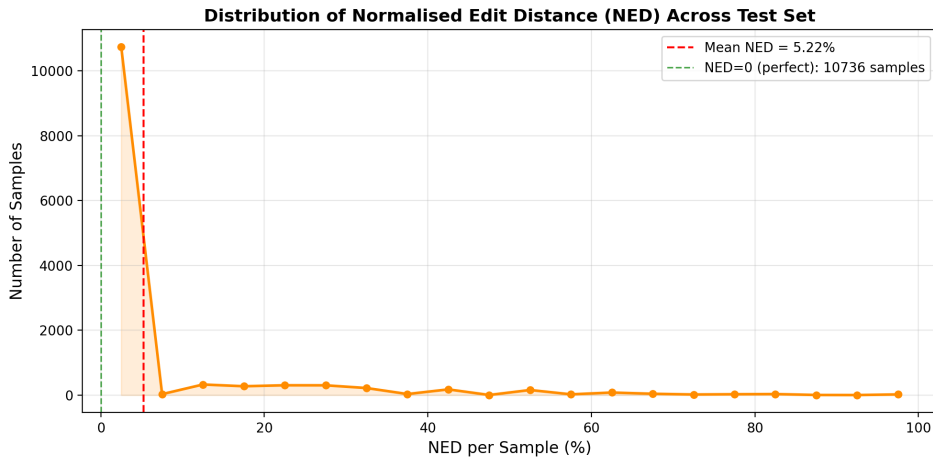


Figure 5.2: Distribution of Normalised Edit Distance (NED) scores across the test set.

Similarly, the cumulative accuracy analysis shows a steep and consistent improvement as the tolerance for character-level errors increases. At zero tolerance ($K = 0$), 83.5% of the predictions are exactly correct, indicating a strong baseline performance. When a tolerance of one character error is allowed ($K = 1$), the accuracy rises significantly to 92.4%, and further increases to 98.1% by $K = 3$. Overall, the accuracy reaches 96.0% within just two character errors, demonstrating that the majority of model predictions are very close to the ground truth.

This trend indicates that most of the model’s errors are minor, typically involving only one or two incorrect characters, rather than large or completely incorrect predictions. Such behavior suggests that the model is generally reliable and rarely produces catastrophic errors.

From an application perspective, this is a particularly strong result. In practical OCR systems integrated with downstream natural language processing tasks, small character-level errors are often recoverable through post-processing techniques such as spell-checking or language model correction. Therefore, even when the model does not achieve perfect predictions, its outputs remain highly usable and can be refined further with minimal effort.

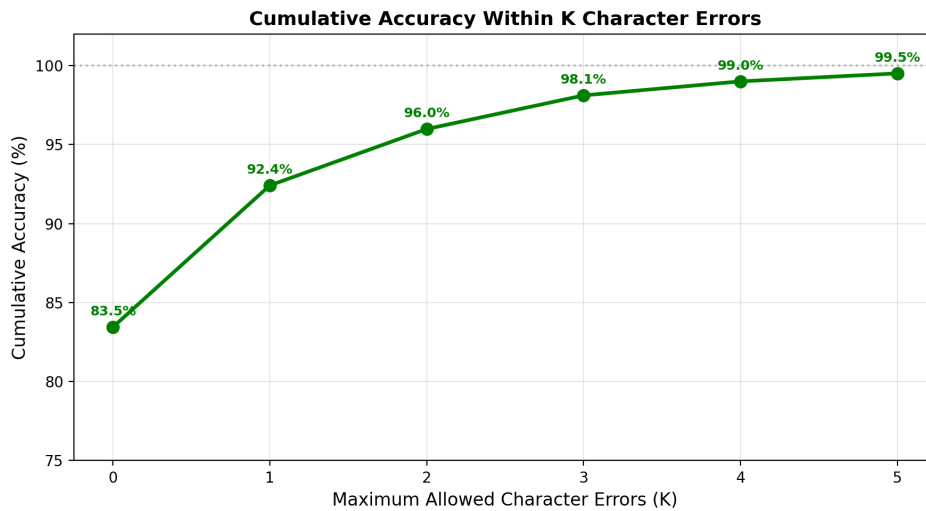


Figure 5.3: Cumulative accuracy within K character errors.

The evaluation of model performance across varying word lengths indicates that exact match accuracy remains relatively stable, ranging between 82% and 85% for words comprising 2 to 11 characters. In contrast, the Character Error Rate (CER) exhibits a pronounced decline as word length increases. This inverse relationship reflects the reduced proportional impact of individual character errors in longer sequences, where isolated mistakes contribute less significantly to the overall error metric.

A consistent trend is observed for shorter to mid-length words (lengths 1–9), where CER decreases substantially from approximately 29% to around 4%. This behavior is expected, as longer words provide increased contextual and structural information, enabling more reliable pattern recognition. Conversely, shorter words, particularly those consisting of one or two characters, present limited contextual cues and are therefore more susceptible to misclassification.

Variations in performance at higher word lengths, specifically at lengths 13 and 15, can be attributed primarily to limited sample sizes. With only 34 and 14 samples respectively, these categories are highly sensitive to the influence of a small number of difficult instances, resulting in noticeable fluctuations in both accuracy and CER. A similar phenomenon is observed at length 14, where accuracy reaches 89.5% based on a sample size of 38, likely reflecting a relatively less challenging subset of data rather than a consistent performance improvement.

Performance degradation for very short words is particularly evident at length 1, where the dataset contains only 24 samples. In addition to the limited sample size, the inherent visual ambiguity of isolated Devanagari characters contributes to this effect. Characters combined with vowel diacritics (matras) often exhibit similar visual structures, and in the absence of surrounding context, the model faces increased difficulty in disambiguating such cases.

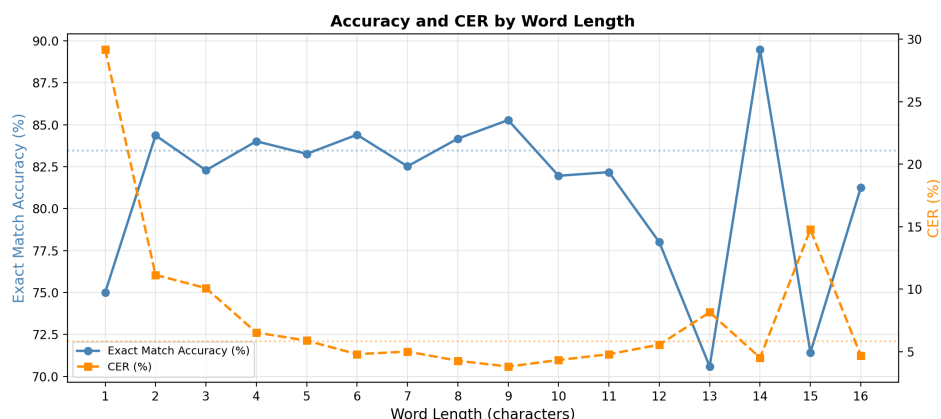


Figure 5.4: Exact Match Accuracy and CER as a function of word length.

Overall, The model achieves a CER of 5.82%, WER of 16.55%, NED of 5.22%, exact-match accuracy of 83.45%, and 1-NED of 94.78%, collectively demonstrating strong character-level recognition with moderate word-level precision.

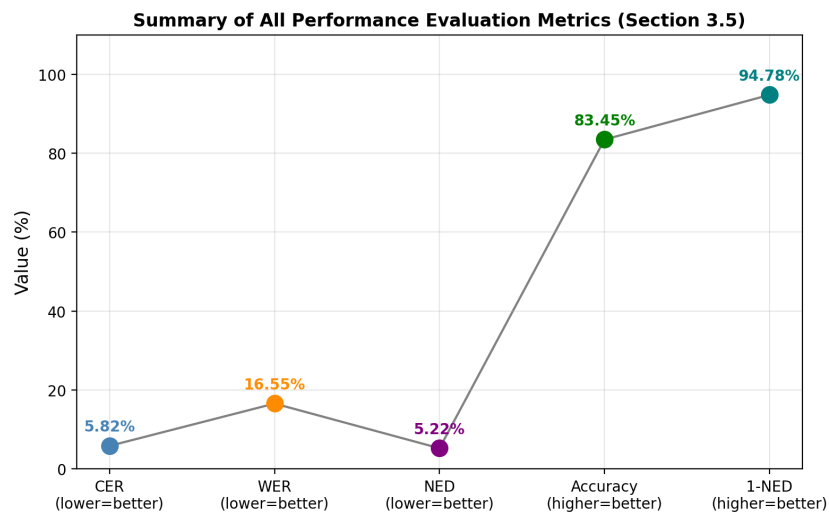


Figure 5.5: Summary of all performance evaluation metrics (Section 3.5).

5.3 Software Development Approach

Agile development methodology is a quick and flexible iterative development methodology for software development. It divides work into manageable chunks without requiring a lot of advance planning. The needs and scope of the project are established early on, and iterations are planned. A functioning product demo is the end result of each iteration, which includes all phases of the software development life cycle: planning, requirements analysis, design, coding, and testing. By iteratively improving the product in response to feedback, this methodology lowers project risk and speeds up delivery overall.



Figure 5.6: Agile Development Model

5.3.1 Agile Sprints

5.3.1.1 Sprint 1: Dataset Acquisition and Initial Exploration

Duration: Mid-December, Weeks 1–2

Goal: Identify and acquire a suitable dataset for handwritten Nepali word recognition and assess the feasibility of a character-level recognition approach.

Key Activities:

- Surveyed publicly available Devanagari handwriting datasets and evaluated their suitability for word-level Nepali OCR.
- Acquired a Devanagari Character Dataset and manually curated an additional matra (diacritic) dataset to supplement underrepresented character classes.
- Implemented and tested an initial character-level recognition pipeline to assess baseline feasibility.

- Identified critical limitations of the character-level approach, including low accuracy and inadequate representation of compound characters and matras.
- Collectively decided to abandon the character-level approach in favour of word-level recognition based on evaluation outcomes.

Outcomes:

- Character-level approach evaluated and rejected on accuracy grounds.
- Decision made to adopt word-level recognition using a dedicated word-image dataset.
- Clear direction established for Sprint 2.

5.3.1.2 Sprint 2: Dataset Preparation and Pre-processing Pipeline

Duration: Late December – Early January, Weeks 3–5

Goal: Acquire the IIIT-HW-Dev word-level dataset, parse it into structured image–label pairs, and build a robust image pre-processing pipeline.

Key Activities:

- Obtained the IIIT-HW-Dev dataset from Kaggle, comprising 95,380 segmented handwritten word images with Unicode labels, and parsed it into training (69,802), validation (12,713), and test (12,865) splits.
- Identified 108 unique Devanagari characters across the dataset, with a maximum label length of 23 characters.
- Designed and implemented a multi-step image pre-processing pipeline comprising grayscale conversion, Gaussian denoising, adaptive binarisation, morphological closing, deskewing, tight cropping, and letterbox padding to 128×128 pixels.
- Pre-cached all processed images to disk to avoid repeated computation during training.
- Built and validated a custom character-level tokenizer with a 112-token Devanagari vocabulary including four special tokens (`<pad>`, `<s>`, `</s>`, `<unk>`).

Outcomes:

- Fully parsed and split IIIT-HW-Dev dataset ready for training.
- Pre-processing pipeline implemented, validated, and cached.
- Custom tokenizer built and smoke-tested on Devanagari word samples.

5.3.1.3 Sprint 3: Model Architecture Design and Initialisation

Duration: January, Weeks 6–7

Goal: Design and initialise the encoder–decoder model architecture and configure the training infrastructure.

Key Activities:

- Selected microsoft/trocr-base-handwritten as the source of the pre-trained ViT encoder, leveraging its existing fine-tuning on handwritten document images.
- Designed a lightweight custom BERT-based decoder with 2 transformer layers, 4 attention heads, hidden size 256, and FFN intermediate size 512 with approximately 4 million parameters that were sized appropriately for the training corpus.
- Integrated the encoder and decoder into a single end-to-end trainable model using HuggingFace’s VisionEncoderDecoderModel framework.
- Configured cross-attention between encoder and decoder, token special IDs, and the data loading pipeline with on-the-fly training augmentation (Gaussian blur, random affine, colour jitter, random perspective, and random erasing).
- Set up mixed-precision training (fp16), gradient accumulation over 4 steps, and automatic checkpoint uploading to Kaggle every 2 epochs.

Outcomes:

- Full model architecture designed and initialised.
- Training infrastructure configured and validated on a small batch of training data.
- Data pipeline with augmentation confirmed operational.

5.3.1.4 Sprint 4: Phase 1 Training – Decoder-Only

Duration: January – February, Weeks 8–10

Goal: Train the decoder in isolation with the encoder frozen to establish baseline Devanagari text generation capability without disturbing pre-trained visual features.

Key Activities:

- Froze all ViT encoder parameters and trained only the decoder using AdamW with learning rate 5×10^{-4} and weight decay 10^{-3} .
- Applied label-smoothed cross-entropy loss with smoothing coefficient 0.1 and gradient clipping at norm 1.0.
- Monitored validation CER and WER every 2 epochs and saved the best-performing checkpoint automatically.
- Observed steady reduction in validation CER across training epochs, confirming that the decoder was successfully learning Devanagari character generation patterns.

Outcomes:

- Stable decoder-only baseline established.
- Best Phase 1 checkpoint saved for use in Phase 2.
- Validation CER and WER trends reviewed collectively to inform Phase 2 training configuration.

5.3.1.5 Sprint 5: Phase 2 Training – Full Fine-Tuning

Duration: February – March, Weeks 11–14

Goal: Unfreeze the ViT encoder and fine-tune the entire model end-to-end to adapt visual features to handwritten Devanagari and further reduce CER.

Key Activities:

- Unfroze the encoder and applied differential learning rates: 5×10^{-7} for the encoder and 5×10^{-5} for the decoder, to prevent catastrophic forgetting of pre-trained visual representations.
- Continued training with `CosineAnnealingWarmRestarts` scheduler ($T_0 = 10$, $\eta_{\min} = 10^{-8}$) across multiple Kaggle sessions, resuming from the latest checkpoint each time.
- Iteratively reviewed validation CER and WER at the end of each session and adjusted hyperparameters accordingly.
- Achieved a best validation CER of 7.26% over the course of full fine-tuning.
- Applied early stopping with patience of 10 validation rounds to prevent overfitting.

Outcomes:

- Best model checkpoint achieving 6.68% CER saved for evaluation.
- Full training completed across 52 epochs with early stopping.
- Model confirmed stable and non-overfitting based on train-validation loss gap monitoring.

5.3.1.6 Sprint 6: Evaluation and Inference

Duration: March, Weeks 15–16

Goal: Conduct final evaluation on the held-out test set, implement single-image inference with confidence scoring, and prepare the system for demonstration.

Key Activities:

- Loaded the best model checkpoint and evaluated it on the full test set of 12,865 word images using beam search (`num_beams = 4`).
- Computed CER, WER, and Exact Match Accuracy on the test set after Unicode NFC normalisation of all predicted and ground-truth strings.
- Implemented single-image inference with test-time augmentation (TTA) across four image variants and confidence scoring from beam sequence log-probabilities.
- Validated predictions qualitatively on sample test images and batch inference grids.
- Compiled training curves, CER history, and sample prediction outputs for inclusion in the project report.

Outcomes:

- Final test set CER, WER, and Exact Match Accuracy reported.
- Single-image inference pipeline operational with confidence scoring.
- System ready for demonstration and submission.

5.4 Model Deployment

This section describes the deployment of the developed Nepali Word Digitizer system, including frontend implementation, backend services, and integration of the trained model for real-time inference.

5.4.1 Frontend Development

The frontend of the system was developed using HTML, CSS, and JavaScript, providing a clean and responsive single-page web interface without relying on any frontend framework. Key frontend components included:

- A drag-and-drop upload zone and file browser for uploading handwritten images in PNG, JPG, or JPEG formats.
- A live image preview displayed immediately after upload for user confirmation.
- A recognition result display showing the recognized Unicode Nepali word in large Devanagari text.
- A character-level breakdown panel showing each predicted character with its confidence score as a percentage and a visual progress bar, derived from per-token transition probabilities computed during beam search decoding.
- An alternative predictions panel displaying the top 3 beam search candidates ranked by sequence log-probability score. Each candidate is clickable, allowing the user to select a more appropriate prediction if the top result is incorrect. Upon selection, the main display and character-level confidence panel update to reflect the chosen candidate.
- A pipeline log panel displaying processing metadata such as model name, device, number of beams, and image dimensions.
- Options to download the recognized text as a .txt file or copy it directly to the clipboard.
- Status messages and error notifications for invalid uploads or failed inference.

Screenshots of the user interface are provided in ??.

5.4.2 Backend Development

The backend API was built using FastAPI, a modern Python web framework, and served the frontend directly. The backend development was primarily focused on the implementation and deployment of the deep learning model and its supporting components:

- **TrOCR Model:** A Vision Encoder-Decoder model was implemented for handwritten Nepali word recognition. The architecture combines a Vision Transformer (ViT) encoder, pre-trained on Microsoft’s TrOCR base handwritten model, with a custom lightweight BERT decoder consisting of 2 layers, 4 attention heads, and a hidden size of 256. The model was fine-tuned on handwritten Nepali word images using a custom Nepali tokenizer trained on Devanagari vocabulary.
- **Preprocessing Pipeline:** A dedicated preprocessing module was implemented to prepare input images before inference. The pipeline performs grayscale conversion, Gaussian blur denoising, adaptive threshold binarization, morphological closing, deskewing, tight cropping to the ink region, and letterbox padding to a fixed 128×128 square before converting back to RGB for the model processor.
- **Configuration Management:** A centralized configuration module was implemented to manage all system parameters including server port, model and tokenizer paths, decoder architecture constants, and generation hyperparameters. Parameters can be overridden at runtime via environment variables without modifying source code.

The trained model and tokenizer were loaded during application startup using PyTorch’s model loading mechanism, keeping them in memory throughout the runtime to ensure fast inference on subsequent requests. Supporting backend functionalities included:

- Loading and initializing the TrOCR model and Nepali tokenizer at startup.
- Receiving and reading uploaded image files via multipart form data with validation for unsupported file types.
- Preprocessing input images through the image preparation pipeline.
- Running model inference using beam search decoding and converting generated token IDs to Unicode Nepali text using the custom tokenizer.

- Computing per-character confidence scores from token transition probabilities for all returned sequences.
- Returning the top 3 alternative predictions, each with its sequence-level log-probability score and per-character confidence breakdown.
- Error handling during file reading, preprocessing, and model inference.

5.4.3 Model Integration

Once the model, preprocessing pipeline, and frontend interface were implemented and tested individually, the final phase involved full system integration:

- Uploaded images are received by the FastAPI backend and passed through the preprocessing pipeline.
- The cleaned image is processed by the TrOCR processor to generate pixel tensors.
- The fine-tuned model performs beam search decoding to generate token IDs, which are decoded into Unicode Nepali text using the custom tokenizer.
- The top 3 candidate predictions are returned alongside their sequence-level log-probability scores. Per-character confidence scores are computed from token transition probabilities for each candidate sequence.
- The recognized word, alternative candidates, character-level confidence breakdown, and pipeline metadata are returned as a JSON response and rendered on the web interface, where users can review and select an alternative prediction if needed.

This end-to-end integration provided a smooth and accessible experience for handwritten Nepali word digitization, combining deep learning, image processing, and web technologies into a single functional application. By following Agile principles, such as iterative development and continuous feedback, we were able to build a reliable and functional word recognizer using modern deep learning and lightweight deployment technologies.

CHAPTER 6

CONCLUSION

6.1 Conclusion

This project successfully developed a Transformer-based OCR system for recognizing handwritten Nepali words in the Devanagari script. The proposed architecture pairs a pre-trained Vision Transformer (ViT-Base) encoder with a lightweight custom BERT-based decoder of approximately 4 million parameters, integrated using HuggingFace’s VisionEncoderDecoderModel framework. The model was trained on the IIIT-HW-Dev dataset comprising 69,802 word images using a two-phase training strategy. Phase 1 trained only the decoder while keeping the encoder frozen, and Phase 2 performed full end-to-end fine-tuning with differential learning rates. The best validation CER of **6.68%** was achieved at epoch 52, and final evaluation on the held-out test set yielded a CER of 5.43%, WER of 16.58%, and Exact Match Accuracy of 83.42%. These results demonstrate that the proposed model effectively handles the structural complexity of Devanagari handwriting, including conjunct consonants, vowel diacritics, and inter-writer variability.

6.2 Limitations

Despite the promising results, the system has the following limitations:

- The model is trained and evaluated on a single dataset (IIIT-HW-Dev), which may limit its generalization to unseen handwriting styles and real-world scanned documents.
- Recognition accuracy decreases for longer words (≥ 13 characters) due to the increased complexity of conjunct consonant sequences in Devanagari.
- The model was trained on Kaggle’s NVIDIA Tesla T4 GPU, which limited exploration of larger architectures such as TrOCR-Large.
- Confidence scores derived from beam search log-probabilities are approximate and may not reliably reflect true prediction certainty for all word forms.

6.3 Future works

Based on the findings of this project, the following works can be achieved in future:

- **Dataset expansion:** Collect and annotate a larger, more diverse Nepali handwriting dataset covering varied writing styles, age groups, and document types to improve generalization.
- **Line and paragraph recognition:** Extend the system by integrating a word segmentation module to enable recognition of full lines and paragraphs rather than isolated words.
- **Data augmentation:** Explore more aggressive augmentation strategies such as elastic distortion and synthetic handwriting generation to further improve robustness.
- **Larger decoder:** Investigate a medium-sized BERT-based decoder (4–6 layers) with sufficient GPU resources to determine whether additional decoder capacity improves accuracy on complex conjunct words.
- **Language model post-processing:** Incorporate a Nepali language model for post-correction of beam search outputs to reduce character-level errors caused by visually similar characters.

APPENDIX A

Raw Dataset

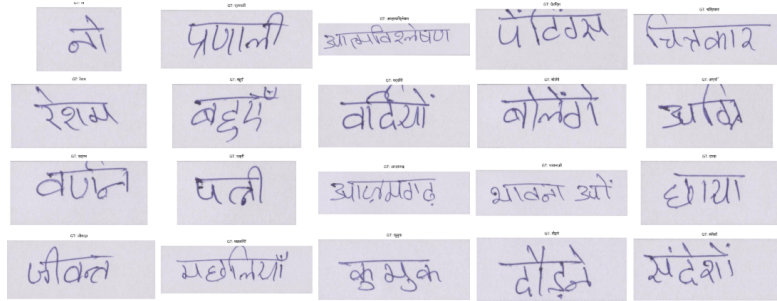


Figure A.1: Sample raw dataset

APPENDIX B

Image Pre-processing Pipeline Details

B.1 Grayscale Conversion and Denoising

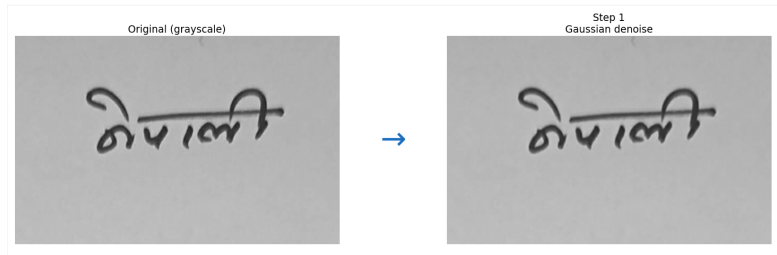


Figure B.1: Grayscale conversion with 3×3 Gaussian blur

B.2 Adaptive Binarisation



Figure B.2: Adaptive Gaussian threshold

B.3 Morphological Closing



Figure B.3: Closing operation

B.4 Deskewing



Figure B.4: Skew correction

B.5 Tight Cropping

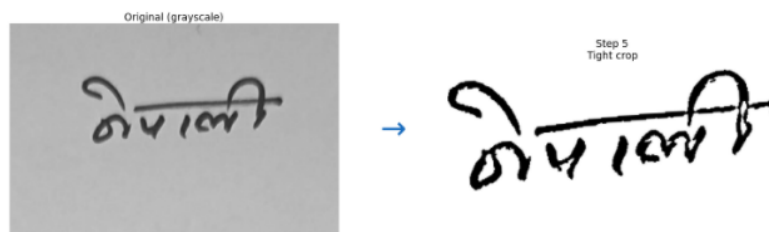


Figure B.5: Bounding box cropping

B.6 Letterbox Padding and Resize

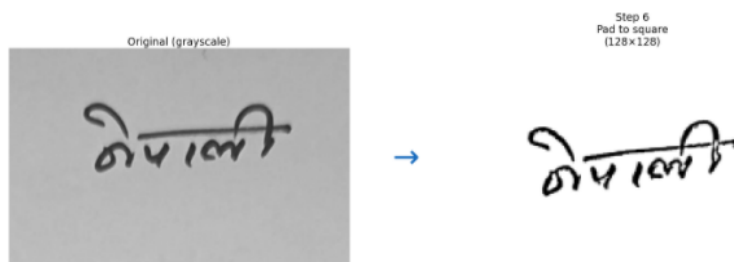


Figure B.6: Resizing to 128×128

B.7 RGB Conversion and Normalisation

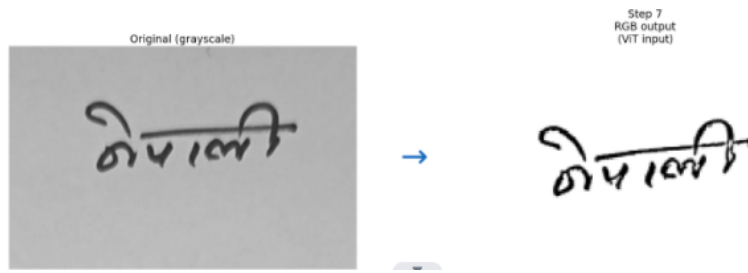


Figure B.7: RGB conversion and normalization

APPENDIX C

User Interface Screenshots

C.1 Main Interface



Figure C.1: Main interface

C.2 Recognized Word

नेपाली हस्तलेखन पहिचान

Handwritten Word Digitizer

TROCR - DEVANAGARI SCRIPT - UNICODE OUTPUT

RECOGNITION ENGINE

UPLOAD IMAGE

Drop image here
or click to browse
PNG - JPG - JPEG



RECOGNIZE CLEAR

RECOGNIZED WORD

कार्यपालिका

क	ा	र	र्	य	प
98%	73%	88%	95%	89%	92%
ा	ल	ि	क	ा	
84%	91%	91%	98%	85%	

ALTERNATIVE PREDICTIONS

#1	कार्यपालिका	score -0.1296
#2	कार्यपालिक	score -0.3828
#3	कार्यपालिया	score -0.4047

PIPELINE LOG

```
model: TroCR Fine-tuned
device: cpu
image_size: 128x128
num_beams: 4
max_length: 27
```

DOWNLOAD .TXT COPY

✓ Recognized "कार्यपालिका" - 11 character(s)

TROCR - DEVANAGARI HANDWRITING RECOGNITION



Figure C.2: Generated word